

Lab 3- Spring Boot MVC and Database Programming

1. Create a new project in IntelliJ. The project should use maven. Name the project Room_Booking.
2. In the dependency window add the following:
 - a. Spring Web
 - b. Thymeleaf
 - c. JPA
 - d. Lombok
 - e. H2
3. Check your POM File and make sure they are similar to the one below. If not add the additional dependencies.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.5</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>roombooking</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>roombooking</name>
  <description>roombooking</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

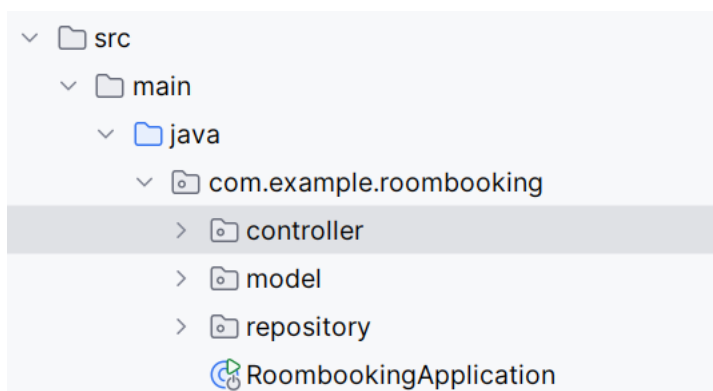
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

4. Create the following folders.



5. Since we will be using H2 as the database, in application properties add the following:

```

1  spring.application.name=roombooking
2  spring.h2.console.enabled=true
3  spring.h2.console.path=/h2-console
4  spring.datasource.url=jdbc:h2:mem:testdb
5  spring.datasource.driverClassName=org.h2.Driver
6  spring.datasource.username=sa
7  spring.datasource.password=
8  spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
9  spring.jpa.hibernate.ddl-auto=update
10

```

6. Inside the model create the reservation class.

```

package com.example.roombooking.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import java.time.LocalDate;

@Entity
public class Reservation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String roomName;
    private LocalDate startDate;
    private LocalDate endDate;
    private String reservedBy;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getRoomName() { return roomName; }
    public void setRoomName(String roomName) { this.roomName = roomName; }

    public LocalDate getStartDate() { return startDate; }
    public void setStartDate(LocalDate startDate) { this.startDate =
startDate; }

    public LocalDate getEndDate() { return endDate; }
    public void setEndDate(LocalDate endDate) { this.endDate = endDate; }

    public String getReservedBy() { return reservedBy; }
    public void setReservedBy(String reservedBy) { this.reservedBy =

```

```
reservedBy; }  
}
```

7. Inside the repository folder, create the RoombookingApplication

```
package com.example.roombooking.repository;  
  
import com.example.roombooking.model.Reservation;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.data.repository.query.Param;  
  
import java.time.LocalDate;  
  
public interface ReservationRepository extends JpaRepository<Reservation,  
Long> {  
  
    // inside ReservationRepository  
    @Query("SELECT COUNT(r) > 0 FROM Reservation r " +  
        "WHERE r.roomName = :roomName " +  
        "AND r.startDate <= :endDate " +  
        "AND r.endDate >= :startDate")  
    boolean existsOverlappingBooking(@Param("roomName") String roomName,  
        @Param("startDate") LocalDate  
startDate,  
        @Param("endDate") LocalDate endDate);  
  
}
```

8. Under templates, create the index template.

```
<!DOCTYPE html>  
<html xmlns:th="http://www.thymeleaf.org">  
<head>  
    <title>Room Reservation</title>  
    <style>  
        body {  
            font-family: Arial, sans-serif;  
            margin: 40px;  
            background-color: #f9f9f9;  
            color: #333;  
        }  
  
        h1 {text-align: center}  
        h2 { color: #2c3e50; margin-bottom: 15px;text-align: center }  
        form {  
            background: #fff;  
            padding: 20px;  
            border-radius: 8px;  
            margin-bottom: 30px;
```

```

        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
        width: 400px;
        margin:auto;
    }
    form input {
        margin: 8px 0;
        padding: 8px;
        width: 95%;
        border: 1px solid #ccc;
        border-radius: 4px;
    }
    form button {
        margin-top: 12px;
        padding: 10px 15px;
        border: none;
        background: #3498db;
        color: #fff;
        border-radius: 4px;
        cursor: pointer;
    }
    form button:hover { background: #2980b9; }
    table {
        width: 100%;
        border-collapse: collapse;
        background: #fff;
        border-radius: 8px;
        overflow: hidden;
        box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    }
    th, td { padding: 12px 15px; text-align: center; }
    th { background: #2c3e50; color: #fff; }
    tr:nth-child(even) { background: #f2f2f2; }
    tr:hover { background: #eaf2f8; }
    .action-btn {
        padding: 6px 12px;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        margin: 0 3px;
    }
    .edit-btn { background: #f1c40f; color: #fff; }
    .edit-btn:hover { background: #d4ac0d; }
    .delete-btn { background: #e74c3c; color: #fff; }
    .delete-btn:hover { background: #c0392b; }
</style>
</head>
<body>

<div th:if="${error}" id="errorBox" style="color: red; margin-bottom:
10px;">
    <p th:text="${error}"></p>
</div>

<script>
    // hide error after 5 seconds
    setTimeout(() => {

```

```

        const box = document.getElementById('errorBox');
        if (box) box.style.display = 'none';
    }, 5000);
</script>

<h2 th:text="${reservation.id} != null ? 'Edit Reservation' : 'Reserve a Room'"></h2>

<form th:action="@${reservation.id} != null ? '/update/' +
${reservation.id} : '/reserve'"
      method="post" th:object="${reservation}">
    <!-- hidden field for editing -->
    <input type="hidden" th:field="*{id}">

    Room: <input type="text" th:field="*{roomName}" required><br>
    Start Date: <input type="date" th:field="*{startDate}" required><br>
    End Date: <input type="date" th:field="*{endDate}" required><br>
    Reserved By: <input type="text" th:field="*{reservedBy}" required><br>

    <button type="submit"
            th:text="${reservation.id} != null ? 'Update' :
'Submit'"></button>

    <a th:href="@{/}" style="margin-left:10px; text-decoration:none;">
        <button type="button">New Reservation</button>
    </a>
</form>

<h2>Existing Reservations</h2>
<table>

<tr><th>Room</th><th>Start</th><th>End</th><th>By</th><th>Actions</th></tr>
    <tr th:each="r : ${reservations}">
        <td th:text="${r.roomName}"></td>
        <td th:text="${r.startDate}"></td>
        <td th:text="${r.endDate}"></td>
        <td th:text="${r.reservedBy}"></td>
        <td>
            <!-- Edit will reload same form pre-filled -->
            <form th:action="@{/edit/{id}(id=${r.id})}" method="get"
style="display:inline;">
                <button type="submit" class="action-btn edit-
btn">Edit</button>
            </form>
            <!-- Delete remains separate -->
            <form th:action="@{/delete/{id}(id=${r.id})}" method="post"
style="display:inline;">
                <button type="submit" class="action-btn delete-btn"
                    onclick="return confirm('Are you sure you want to
delete this reservation?');">
                    Delete
                </button>
            </form>
        </td>
    </tr>
</table>

```

```
</table>

</body>
</html>
```

9. Finally, create the controller inside the controller package.

```
package com.example.roombooking.controller;

import com.example.roombooking.model.Reservation;
import com.example.roombooking.repository.ReservationRepository;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
public class ReservationController {
    private final ReservationRepository repository;

    public ReservationController(ReservationRepository repository) {
        this.repository = repository;
    }

    // Home page - show list + empty form
    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("reservations", repository.findAll());
        model.addAttribute("reservation", new Reservation()); // empty for
new adds
        return "index";
    }

    // Add new reservation
    // @PostMapping("/reserve")
    // public String reserve(@ModelAttribute Reservation reservation) {
    //     repository.save(reservation);
    //     return "redirect:/"; // PRG pattern - clears form after submit
    // }

    @PostMapping("/reserve")
    public String reserve(@ModelAttribute Reservation reservation, Model
model) {
        if (repository.existsOverlappingBooking(reservation.getRoomName(),
            reservation.getStartDate(),
            reservation.getEndDate())) {
            model.addAttribute("error", "Room is already booked for these
dates!");
            model.addAttribute("reservations", repository.findAll());
            model.addAttribute("reservation", reservation); // keep filled
form
            return "index";
        }

        repository.save(reservation);
    }
}
```

```

        return "redirect:/";
    }

    // Load reservation for editing (prefilled in same form)
    @GetMapping("/edit/{id}")
    public String editReservation(@PathVariable Long id, Model model) {
        Reservation existing = repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Invalid
reservation Id:" + id));
        model.addAttribute("reservation", existing); // prefill the form
        model.addAttribute("reservations", repository.findAll()); // still
show table
        return "index"; // reuse the same Thymeleaf template
    }

    // Update existing reservation (same form handles add + update)
    @PostMapping("/update/{id}")
    public String updateReservation(@PathVariable Long id, @ModelAttribute
Reservation reservation) {
        reservation.setId(id); // ensure correct ID is set
        repository.save(reservation); // JPA will update if ID exists
        return "redirect:/";
    }

    // Delete reservation
    @PostMapping("/delete/{id}")
    public String deleteReservation(@PathVariable Long id) {
        repository.deleteById(id);
        return "redirect:/";
    }
}

```

The interface when ran properly would look like the one below:

Reserve a Room

Room:

Start Date:

End Date:

Reserved By:

Existing Reservations

Room	Start	End	By	Actions
------	-------	-----	----	---------